



NOWITNESS

Security Audit Report

Cardano Programmable Asset Tokens: CIP-113 First Module

Date	September 22, 2026
Client	Cardano Foundation
Project	CMTA Reference Profiles (cpt-rwa-ch-de-cmta-reference)
Version	0.1.0

Contents

Disclosure	1
Disclaimer	2
Scope	4
Assessment Overview	5
Assessment Components	6
Security Assessment Framework	7
Executive Summary	8
Code Base	12
Severity Classification	14
Finding Severity Ratings	15
Findings	16
[M 301] Seizure Path Appears to Permit Taking the CIP-68 Reference NFT and Rewriting Token Metadata	17
[L 201] PowerUser credential_hash is not bound to the linked-list node key	22
[L 202] Mint-created security-token UTxOs bypass the CIP-113 inline datum size bound	24
[L 203] security_asset_name is compiled into four validators with no on-chain coherence check	26
[L 204] Registry fields 3 and 4 are not pinned to the compiled transfer-logic hashes at registration	29
[L 205] GlobalState datum caps are per-field, not aggregate	32
[L 206] PauseTransfers can churn the GlobalState UTxO and race admin recovery .	34
[L 207] Authorized seizure applies to script-held tokens without consent, and the resulting accounting divergence is undisclosed	36
[I 101] KYC attestation format relies on an unstated single-purpose-key invariant .	39
[I 102] Genesis can enable KYC without configuring a proof source	40
[I 103] KYC attestation field bindings are not pinned by any test	41
[I 104] Denylist metadata is unbounded	43
[I 105] GlobalState outputs can carry an unnecessary reference script	45
[I 106] RotateAdmin does not rotate CIP-68 metadata custody, and LockUpgrades can make a missed handover unrecoverable	47

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Cardano Foundation** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical security purposes and must not be considered legal, regulatory, financial, or investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as continued testing, monitoring, formal verification where appropriate, and bug bounty programs.

The scope of this audit was limited to the code and materials provided for review. It did not extend to compiler-level artefacts, generated UPLC code, deployment infrastructure, off-chain transaction construction, user interfaces, wallets, dApps, indexers, backend services, or areas beyond the explicitly reviewed codebase. Developing a comprehensive unit-test suite, property-based test suite, or formal verification model was also outside the scope of this engagement.

CIP-113 Base-Layer Dependency

The reviewed contracts are a module layered on top of the CIP-113 programmable-tokens framework. They rely on the base layer's `programmable_logic_base`, `programmable_logic_global`, registry, and issuance components to invoke the module's transfer, third-party, and minting logic at the right moments and to enforce value conservation. This review assessed the module's validators and their integration boundary with the base layer, but did not re-audit the CIP-113 base layer itself. Base-layer behavior was verified against the base-layer source available at the time of review; deploying against a different base-layer revision may invalidate conclusions made in this report.

Administrative and Governance Assumptions

The reviewed design intentionally grants strong administrative capabilities: denylisting, global pause, forced transfers and seizures, supply-cap management, and an irreversible decommission mechanism, exercised by an admin credential and delegated power-user roles. This report assesses the technical behavior of those mechanisms, but does not evaluate whether a given administrative model, issuer policy, governance process, freeze/seizure mechanism, or compliance workflow is legally, commercially, or operationally appropriate. In particular, the profiles are intended to support, not establish or ensure, compliance with any legal or regulatory framework, including the Swiss and German frameworks they reference. Users and integrators should understand the administrative authority granted by the deployment before relying on it.

Deployment and Configuration Assumptions

The security properties described in this report assume that the deployment is initialized correctly and coherently: that compile-time validator parameters are applied consistently across all scripts, that registry fields reference the intended transfer and third-party logic scripts, that genesis state (including KYC configuration, trusted-entity keys, and linked-list bootstraps) is well formed, and that trusted off-chain actors, in particular KYC attestors, manage their keys according to the documented model. Several findings in this report concern exactly this class of assumption. Incorrect initialization, parameter mismatches, or deployment under different configuration assumptions may invalidate conclusions made in this report.

Scope

In scope

- The module's on-chain validators and supporting library code at the audited commit: global state, minting authority, minting logic, transfer logic, third-party transfer logic, power-user list, and denylist scripts, plus the KYC, denylist, CIP-68, and utility libraries.
- Datum/redeemer structures and on-chain invariants (roles and authorization, supply cap accounting, KYC and denylist gating, pause and deactivation semantics, linked-list integrity).
- KYC proof formats (trusted-entity attestations and membership proofs) as verified on chain.
- The integration boundary with the CIP-113 base layer: registration pinning, dispatch of transfer and third-party logic, and seizure mechanics as they interact with the module's checks.

Out of scope

- The CIP-113 base-layer implementation itself (`programmable_logic_base`, `programmable_logic_global`, registry, and issuance components), beyond its documented behavior at the integration boundary.
- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, deployment tooling, KYC attester infrastructure, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals, centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.
- Legal or regulatory compliance assessment of the Swiss and German profiles.

Assessment Overview

From **September 01, 2026** to **September 22, 2026**, **Cardano Foundation** engaged No Witness Labs to conduct a comprehensive security assessment of the **CMTA Reference Profiles (cpt-rwa-ch-de-cmta-reference)** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Review** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual Review

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- Business logic and protocol-specific invariants
- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Cheap spam
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Script output datum
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention
- Stake-credential ownership and authorisation controls
- Value and accounting calculations
- Integer sign and token-quantity validation
- Minimum Ada requirements

Security Assessment Framework

This assessment combined four complementary review layers. No single checklist was treated as exhaustive.

- **No Witness Labs audit baseline** – The review used the internal security baseline maintained by No Witness Labs, covering common vulnerability categories and audit checks. The principal categories applicable to this engagement are presented in the Assessment Components section.
- **Applicable Cardano Improvement Proposals (CIPs)** – CIPs applicable to the protocol's reviewed functionality were identified, and their relevant implementation requirements were assessed for correctness and security. For this engagement these notably include CIP-113 (programmable tokens), CIP-68 (datum metadata standard), and CIP-31 (reference inputs). Unrelated CIPs were not applied indiscriminately.
- **Protocol specification and security invariants** – The implementation was evaluated against the protocol's intended design, supplied documentation, and stated invariants, including authorisation rules, asset preservation, state transitions, supply-cap accounting, compliance gating, and other protocol-specific security properties.
- **Adversarial and economic analysis** – The review considered unexpected or attacker-controlled transaction constructions, authorisation bypasses, value extraction, state manipulation, denial-of-service conditions, economic abuse, and edge cases arising from interactions among validators, roles, and the CIP-113 base layer.

Executive Summary

Project Overview

The audited repository provides Aiken (Plutus V3) contracts implementing on-chain primitives for programmable asset tokens, designed as reference profiles for use cases based on the CMTA framework and supporting the implementation of Swiss and German legal requirements. The primitives include KYC-gated transfers, denylisting, global pause, forced transfers and seizures, supply caps, role-based permissions, and an irreversible decommission mechanism.

The contracts are built as a module on top of CIP-113 (programmable tokens): security tokens are held at programmable-base smart-wallet addresses, and the base layer dispatches every transfer, third-party action, and mint or burn to this repository's transfer logic, third-party transfer logic, and minting authority scripts, which enforce the compliance rules.

Extended Overview

Design Overview

The module consists of a small set of cooperating validators:

- `global_state`: A single `GlobalState` UTxO, identified by an NFT, carrying the protocol's central configuration and state: the admin credential, KYC requirement flags, trusted-entity keys, membership root, supply cap (`mintable_amount`), pause flag, upgrade lock, deactivation flag, and security metadata.
- `minting_authority`: The module's registration and mint/burn authority. It registers the token policy in the CIP-113 registry (pinning the CIP-68 reference NFT owner and registry structure), and validates `MintBurn` actions against the supply cap, KYC status, and denylist absence of destinations.
- `transfer_logic_script` and `third_party_transfer_logic_script`: The per-policy scripts the CIP-113 base layer invokes for ordinary transfers and for third-party (forced-transfer / seizure) actions. They enforce the pause gate, deactivation, denylist absence, and KYC proofs per sender and receiver.
- `power_users` and `denylist`: Two on-chain linked lists. Power-user nodes carry independently grantable role flags; presence in the denylist is the sanction, and absence is proven with a covering-node proof.

Roles and Responsibilities

- Admin: the `admin_credential_hash` held in the `GlobalState` datum. Controls the power-user list itself (add, remove, modify), the security metadata, and the irreversible deactivation switch. Rotatable, so the master key can be replaced without redeploying.
- Power user: a node in the power-users linked list, carrying independently grantable flags: `is_admin` (add and remove denylist entries), `can_mint` (mint new tokens, up to the remaining supply cap), `can_burn` (burn tokens, restoring headroom under the cap), `can_pause` (pause and unpause all standard transfers), and `can_force_transfer` (seize tokens, moving them to a vetted address).

- Trusted entity: an off-chain KYC attester whose Ed25519 verification key is listed in the GlobalState datum. It signs the KYC proofs that transfers present. Not an on-chain role: it never submits transactions, it only signs attestations.
- Holder: any wallet or script holding the token. Must be absent from the denylist and, when the corresponding flag is set, must present a valid KYC proof (or prove membership of the allowlist tree).

Lifecycle of Assets

- Initialisation: An issuer initialises the protocol (GlobalState, power-user and denylist lists), registers the token policy in the CIP-113 registry, then grants power-user flags to the operators who need them.
- Transfer: A transfer must clear the pause gate, then per sender and per receiver: denylist absence, plus a valid KYC proof whenever `requires_sender_kyc` / `requires_receiver_kyc` is set. The two are toggled independently.
- Mint and burn: Minting is capped; burning returns headroom to the cap. Mint and burn remain available during a pause so the register stays correctable. Burning during a pause requires the forced-transfer path and both `can_burn` and `can_force_transfer`.
- Forced transfer / seizure: An operator holding `can_force_transfer` can move tokens to a vetted, non-denylisted destination. Forced transfers intentionally remain available while transfers are paused, so enforcement is not blocked by a pause.
- Decommissioning: Deactivation is one-way and requires a paused protocol first. Dismantling the two linked lists (`Deinit`) is only possible afterwards and is equally irreversible.

Supporting Infrastructure

The deployment depends on the CIP-113 base layer (programmable-logic base and global validators, registry, and issuance policy), off-chain KYC attestors that sign attestation payloads or maintain the membership Merkle tree, and deployment tooling that applies coherent compile-time parameters across the validators and initialises genesis state correctly.

Risks and Dependencies

The security of the deployment rests on the CIP-113 base layer behaving as documented at the pinned revision, on coherent deployment configuration (several

findings in this report concern parameter and registration coherence), on the admin and power-user key custody model, and on trusted-entity keys being used exclusively for the documented KYC attestation format. The administrative model is intentionally powerful; its appropriateness for a given legal or regulatory context is outside the scope of this review.

Code Base

Repository

<https://github.com/cardano-foundation/cpt-rwa-ch-de-cmta-reference>

Branch

main

Audited Commit

06a8059d21c70624b694a73dcf54d566d9b79695

Fixes Verified Commit

7dbc40051a7e840f09ece77d9bf187cef768fd14

Remediation verification used Aiken 1.1.23 at this revision. All 363 tests passed with `aiken check --trace-level compact`, including the F201, F104, and F105 fix regressions, F301's protected-token seizure regression and positive control, and the F102 operator-destination receiver-KYC exemption. This verifies the reviewed fixes at validator-test level; it is not deployment verification. The file hashes below identify the original audited revision.

Files Audited

Files	SHA256 Checksum
validators/denylist.ak	916072a0d8cb65d2d50a011ab541db818a77a23d7ddc3a9c3ccc4b21da9d6048
validators/global_state.ak	398c11d4a94e6ff31395a82ed087f4fbf4ad1d057f12dc2969c2684e2bacef78
validators/minting_authority.ak	076f15b2b46a34e8330028503ae605eb608669835d14e2830bcea99ae93942c7
validators/minting_logic_script.ak	4d104a5f34c55f53a822a2cd0e750a1530942f46f72ab9e6a1ed3f75ccf7bad4
validators/power_users.ak	be384d88e7d1c62c029fda852fc59d0a9093582372507218f8d458ca6cc6ff9d

Files	SHA256 Checksum
validators/regression.ak	f607aca8748b1127c1239b82909fafb6242869a300958595b608a0ec052eecb3
validators/third_party_transfer_logic_script.ak	f03d24c0fda6f66b863557d993c1392c11a319a34735fafe9a85c6829321fb70
validators/transfer_logic_script.ak	6d78a8a6b09037a200a6d17e6691354881653df7e79bab15704411aac69d5a12
lib/cip68.ak	d17e949a365d9205e044a76bbb0203dd889b322072dcfaaa7f78ae9e18c477ac
lib/compliance.ak	2c02372ec3391601fbc500548e4f79c97de7c620524eef7a8476aab1eaf84c57
lib/constants.ak	2e6b3d0671cbb20916ff73d58fb9fb30fa5591e32dfa2160ca539a70e7dc041f
lib/denylist/absence.ak	aaad34d47b26abd5e6b3e30b3d8aa57b1a0933058e25a96a0db6ae525bcd596a
lib/kyc/verify.ak	6ebcc4eb5dc48a206a1c3fc681b70e0ee2f894160295f57bd518cd781b89d607
lib/tests.ak	e52a5ba58b6a676a5fa800bb63b2f2c28434a4b4f0a63070bc418a5b5037c30b
lib/types/cip68.ak	9d8291f1568153de712e82cb2e91404d7db302f97966072f57118066c64a5db6
lib/types/denylist.ak	9ccf5e06ce4e1fec1d125508c83d8f9f68d271ae8c0d425fe6845f5d7efe4c0d
lib/types/global_state.ak	880f8b092e24b397c561f8062072411f387194baeccd94191d5504d1825aa75c
lib/types/kyc_proof.ak	bbe4ce12bb54b0380f4f23870afb2ff56258195c6be855c7c5ac458680771daf
lib/types/minting_authority.ak	ff5bed68bf9b3f0fc1811bc7f1fbf349a7768bae391bccbd21b26429659bcb79
lib/types/power_users.ak	7db2717c6932fec4ae5af7a573e0fc991498b540b75d87bf83965a13605decbf
lib/types/security/bafin.ak	cda1b48a8801233110ed83d5a802c08c7c5d0a1079f21b54ae7db16d3caef101
lib/types/third_party_transfer_logic_script.ak	7939643789d89c02b08827dd42ae3304f41d0a24f92b2cd5cd1c2063d50d1b33
lib/types/transfer_logic_script.ak	aef5965fbc400bcfa64b74c9601659f176a39fe48d356ce1d234ad94c7e19828
lib/utls.ak	37c3ed2a42c02584ef6e755adfe5f81a935b3d13eae0e502af4e489f5def86d

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Count
	5	Critical	0
	4	High	0
	3	Medium	1
	2	Low	7
	1	Informational	6
Total Findings			14

Findings

ID	Title	Severity	Status
301	Seizure Path Appears to Permit Taking the CIP-68 Reference NFT and Rewriting Token Metadata	Medium	Resolved
201	PowerUser credential_hash is not bound to the linked-list node key	Low	Resolved
202	Mint-created security-token UTxOs bypass the CIP-113 inline datum size bound	Low	Acknowledged
203	security_asset_name is compiled into four validators with no on-chain coherence check	Low	Acknowledged
204	Registry fields 3 and 4 are not pinned to the compiled transfer-logic hashes at registration	Low	Acknowledged
205	GlobalState datum caps are per-field, not aggregate	Low	Acknowledged
206	PauseTransfers can churn the GlobalState UTxO and race admin recovery	Low	Acknowledged
207	Authorized seizure applies to script-held tokens without consent, and the resulting accounting divergence is undisclosed	Low	Acknowledged
101	KYC attestation format relies on an unstated single-purpose-key invariant	Informational	Acknowledged
102	Genesis can enable KYC without configuring a proof source	Informational	Acknowledged
103	KYC attestation field bindings are not pinned by any test	Informational	Acknowledged
104	Denylist metadata is unbounded	Informational	Resolved
105	GlobalState outputs can carry an unnecessary reference script	Informational	Resolved
106	RotateAdmin does not rotate CIP-68 metadata custody, and LockUpgrades can make a missed handover unrecoverable	Informational	Acknowledged

[M 301] Seizure Path Appears to Permit Taking the CIP-68 Reference NFT and Rewriting Token Metadata

	Level	Severity	Status
	3	Medium	Resolved

Category: Authorization Model / Data Integrity

Affected Files:

- `validators/third_party_transfer_logic_script.ak`
- `validators/minting_authority.ak`

Historical scope: The description and affected-code references below concern vulnerable revision `8ee708c813229f76b71d32280a7b7d43bc4b5530`, the parent of the fix commit. The original audited revision `06a8059d21c70624b694a73dcf54d566d9b79695` already contains the fix.

Affected Code: `validators/third_party_transfer_logic_script.ak:115-138` (destination fold, scoped to `security_asset_name`) and `:184-190` (final conjunct block, where the missing exclusion belongs)

Related: `validators/minting_authority.ak:553-582` (the registration pin this defeats), `:540-546` (comment describing a protection that does not hold on this path)

Description

Under CIP-68 the reference NFT is the metadata pointer. The record for `(333)SEC` is the datum on the UTxO holding `(100)SEC`, and both sit under the same issuance policy because the standard requires it. Whoever holds the NFT therefore controls the token's published metadata.

The module pins the NFT owner to `admin_credential_hash` at registration, making the admin the metadata authority by construction (`validators/minting_authority.ak:553-582`). Reading both layers together, no check was found that would stop a power user holding only `can_force_transfer` from moving that NFT and writing a new datum on it in a single transaction.

Root Cause

In this repository

All of it sits in the `withdraw` handler of `validators/third_party_transfer_logic_script.ak`.

- The destination fold selects outputs by policy and asset name. An output carrying only the reference NFT holds zero `security_asset_name`, takes the `else` branch, and never enters `outputs_user_credentials`.

```

1  if quantity_of(
2      output.value,
3      expected_issuance_policy_id,
4      security_asset_name,
```

aiken

```

5  ) > 0 {
6    expect Some(Inline(stake_cred)) = output.address.stake_credential
7    ...
8  } else {
9    result
10 }

```

(validators/third_party_transfer_logic_script.ak:115-138)

A credential absent from that list is never paired with a `ThirdPartyDestinationAction`, so no denylist proof and no KYC proof is demanded for it. It is not a party missing a proof, it does not exist as far as the validator is concerned.

- Nothing in the final conjunct block compensates. Five checks, none of them about asset names.

```

1  and {
2    gs_datum.deactivated == False,
3    at_least_one_seized_input,
4    all_destinations_ok,
5    power_user_data.can_force_transfer,
6    must_be_signed_by_credential(self, power_user_node_key),
7  }

```

aiken

(:184-190)

`at_least_one_seized_input` is itself scoped to `security_asset_name` (:167-182), so it does not stop the reference NFT UTxO being added as an extra input.

- The control meant to protect the NFT lives in another validator and runs once. `reference_nft_output_is_pinned` is defined at `validators/minting_authority.ak:553` and called once at :288, inside the `RegisterMint` branch. Nothing re-asserts owner-equals-admin afterwards, and the README confirms the pin “applies at registration only” (README.md:301-303).

In the base layer

Not defects. Documented behaviour, and the conditions that make the gap above reachable.

- The seizure path skips owner consent. `authorised_stake_cred` appears 0 times in `cip113-programmable-tokens/validators/programmable_logic/third_party.ak` against 2 times in `.../transfer.ak`, which `documents/security/security-fixes.md:658` also records. The transfer path asks the NFT’s owner, the seizure path does not.
- Seizure acts on a whole policy, not an asset name.

```
1 let minted_or_burned = self.mint |> value.tokens(policy_id) aiken

( cip113-programmable-tokens/validators/programmable_logic/third_party.ak:47, and
  assets_split_at(input_assets, policy_id) at the same file :216 )
```

So both asset names under the issuance policy are in scope for the base layer while only one is in scope for the module.

- Datum identity is checked on paired outputs only. The paired branch requires `output.datum == input.datum` (`.../third_party.ak:192-195`). The unpaired branch checks an inline stake credential and `is_seizable_output_shape_bounded`, which bounds datum size and not content (`.../third_party.ak:153-158`, `cip113-programmable-tokens/lib/assets.ak:273-284`). Neither branch constrains how much of the acted policy a paired output retains.
- Companion assets are explicitly the module's job. Doc 03 §1 puts CIP-68 metadata out of scope, since CIP-113 applies control at the policy level while CIP-68 operates at the token level, and states that "companion assets are therefore handled by substandards layered on top". Doc 09:134-141 repeats it: "Companion assets (CIP-68 / CIP-102) are your responsibility, not the framework's ... Transferring them is allowed".

The mechanism CIP-113 names for this is `protected_prefixes`, specified in operative detail in doc 03 §2.2 down to the prefix bytes for label 100. It is absent from the base-layer source at `9db7e06`, so the prescribed route is unavailable. That is a base-layer gap worth raising, and it does not move the obligation.

Attack Vector

```
1 inputs:
2   [0] PLB addr=Operator datum=D_op { SEC: 100 } ← would satisfy
   at_least_one_seized_input
3   [1] PLB addr=Admin datum=D_meta { REF: 1 } ← the target
4
5 outputs:
6   [0] addr=Operator datum=D_op { } ← paired, same address and datum, empty
7   [1] addr=Admin datum=D_meta { } ← paired, same address and datum, empty
8   [2] addr=Operator datum=D_op { SEC: 100 } ← vetted destination, operator's own
   proof
9   [3] addr=? datum=D_NEW { REF: 1 } ← unpaired, not a destination, datum
   free
```

Conservation appears satisfied because `{SEC:100, REF:1}` is present across outputs [2] and [3]. The operator supplies input [0] from their own holding, so the only destination proof needed is one they can produce for themselves.

Three further constraints apply to this shape. `inputs` is ledger-sorted by output reference, so the pairing is not freely chosen and the outputs would need ordering to match. The base layer's `outputs_start_idx` prefix also counts toward conservation, so output [3] can sit either side of it. And the per-pair lovelace ratchet applies to output [1], which must carry at least the admin UTxO min-ADA.

Impact

The transaction is available to a power user holding `can_force_transfer`, a role the admin grants explicitly and revokes by removing the operator node.

- The metadata record ends up operator-written but canonical, not forged. Only one `(100)SEC` can exist (`validators/minting_authority.ak:519, :262-271`), so a correct CIP-68 reader takes that datum as the token's metadata. There is no authentic copy to compare against.
- A separation of duties the protocol codes deliberately is broken. `can_force_transfer` is kept distinct from admin so a compliance function can be delegated without handing over control of the protocol (`README.md:97-99`). This hands the lesser role authority over what the design pins to the greater one.
- The weaker of the two metadata stores is the one third parties read. `security_info` is admin-gated, size-capped and limited to one field per spend. The CIP-68 datum is protected only by NFT possession.
- Nothing on chain flags it. The variant that returns the NFT to the admin leaves no ownership anomaly, so noticing it would take a datum comparison over time.

No funds are at risk. Supply, balances, denylist and KYC gates stay untouched, and the NFT could presumably be seized back, though metadata written in the meantime is published from the moment the transaction settles.

The role may also be weaker than assumed. `lib/utils.ak:23-46` documents a case where a role credential naming a permissive script is satisfiable by anyone, and names `transfer_logic_script`, whose `withdraw-0` requires no signature, as the worst instance.

Remediation

Confine the seizure path to `security_asset_name`, so the reference NFT cannot appear in a seizure transaction at all. Two directions:

- Allowlist by asset name: Reject any input or output holding an asset under `expected_issuance_policy_id` other than `security_asset_name`. No new parameter, and it covers any future companion asset under the same policy. Recommended.
- Block `reference_asset_name` only: Needs it as a new validator parameter, and reopens the gap if another asset is minted under the same policy later.

Resolution

The fix adds `no_protected_tokens_spent`, which rejects seizure transactions that spend protected CIP-68 tokens under the issuance policy. It is already included in the original audited revision. At verified revision `7dbc40051a7e840f09ece77d9bf187cef768fd14`, `seizure_cannot_spend_a_protected_prefix_token` and its positive control `seizure_succeeds_when_no_protected_token_is_spent` both passed with Aiken 1.1.23.

Resolved in commit hash `5033daaae58d86d08af7dcc8403ffa9e2fbc17bb`

[L 201] PowerUser credential_hash is not bound to the linked-list node key

	Level	Severity	Status
	2	Low	Resolved

Category: Data integrity / Off-chain indexing

Affected Files:

- `lib/types/power_users.ak`
- `lib/utls.ak`
- `validators/power_users.ak`

Description

Each power-user node has two identity fields: the linked-list node key, derived from the list NFT asset name, and `PowerUser.credential_hash` inside the node datum.

On-chain authority uses the node key. `authenticated_power_user` returns that key to callers, and role checks use it for signature or withdrawal authorization. The `credential_hash` datum field is not read by production validators and is not checked against the node key when a power user is added or modified.

PoCs confirm the behavior: `AddPowerUser` and `ModifyPowerUser` accept a datum whose `credential_hash` differs from the node key, while a node key that differs from the redeemer key traps.

Impact

No on-chain privilege is gained or lost. Authorization remains bound to the linked-list node key.

The impact is register accuracy. A power-user node can authorize credential `B` while its datum reports `credential_hash = A`. No script in this repository reads that field today, so the off-chain-reader impact is hypothetical for now, but indexers, dashboards, or audits that trust the datum field may report the wrong operator identity.

Recommendation

- Bind the field to the node key. Assert `node_data.credential_hash == node_key` in `AddPowerUser`, and assert the same invariant for the new datum in `ModifyPowerUser`.
- Avoid dropping the field unless deployments can absorb the schema change. Removing `PowerUser.credential_hash` changes the datum shape in `plutus.json` and breaks existing deployments, so binding the field is the safer remediation.

Related Findings

- Finding **F102** is also a configuration-integrity hardening note, but for genesis KYC state.

Resolution

`AddPowerUser` and `ModifyPowerUser` now require `PowerUser.credential_hash` to equal the authenticated linked-list node key. The insertion and modification tests that reject mismatched credential hashes passed at verified revision `7dbc40051a7e840f09ece77d9bf187cef768fd14` with Aiken 1.1.23.

Resolved in commit hash `58f9dbd00136e9f7ddb8b8755ddd85b03d33e0c4`

[L 202] Mint-created security-token UTxOs bypass the CIP-113 inline datum size bound

	Level	Severity	Status
	2	Low	Acknowledged

Category: Integration boundary / Seizability

Affected Files:

- `validators/minting_authority.ak`
- `../cip113-programmable-tokens/validators/issuance_mint.ak`
- `../cip113-programmable-tokens/lib/assets.ak`
- `../cip113-programmable-tokens/validators/programmable_logic/datum_size_cost.test.ak`

Description

CIP-113 bounds inline datum size on holder-created programmable-base outputs, including ordinary transfers. The mint path is different: `issuance_mint` only checks that minted token outputs have the programmable-base shape. It rejects datum hashes and reference scripts, but it does not cap inline datum size.

This repository owns the module minting authority that CIP-113 expects to enforce mint-side rules. `verify_mint_destinations` checks the minted value, destination stake credential, KYC status, and denylist absence, but it does not inspect the destination datum.

PoCs confirm both layers. A `MintBurn` action with a 4 KB destination datum passes this minting authority. In the base layer, a seizure of that UTxO passes only when the paired output reproduces the datum, while the holder can spend the same UTxO into `NoDatum` outputs freely.

Impact

A `can_mint` operator can mint security tokens into a programmable-base UTxO with an inline datum that is unbounded by these validators, subject to ledger limits. Supply accounting, KYC, and denylist checks are unaffected.

The enforcement impact is asymmetric. The base layer's `datum_size_cost.test.ak` header documents the practical transaction-size arithmetic: about 13 seizure pairs fit per transaction at 1 KB datums, and only one pair fits at 8 KB datums under the 16,384 byte transaction-size limit. Below near-limit datum sizes, the cost is reduced batching capacity rather than impossibility; near the limit, a position can become practically unseizable.

The party choosing the mint datum is the issuing side, not the holder. Bloat also does not spread through ordinary transfers, because holder-created outputs are bounded and the owner can shrink the datum on spend. The realistic failure mode is an off-chain mint builder that supplies a large datum and a `can_mint` operator that signs without inspecting it.

Recommendation

- Enforce a conservative inline datum size bound inside `verify_mint_destinations` for minted security-token outputs. Requiring `NoDatum` may be too strict because SDKs write a unit datum today; a size bound is the safer compatibility point.
- Keep CIP-68 metadata outputs separate from security-token destination checks. The owner can shrink an oversized CIP-68 datum on spend, so the concern there is update liveness, not permanent immobility.

Related Findings

- Finding **F301** covers the other datum-integrity gap under the same issuance policy: the seizure path does not exclude the CIP-68 reference NFT, so its metadata datum can be rewritten. Both are addressed by keeping CIP-68 metadata outputs out of the security-token paths.

[L 203] `security_asset_name` is compiled into four validators with no on-chain coherence check

Level	Severity	Status
2	Low	Acknowledged

Category: Deployment / configuration

Affected Files:

- `validators/global_state.ak`
- `validators/minting_authority.ak`
- `validators/transfer_logic_script.ak`
- `validators/third_party_transfer_logic_script.ak`

Description

`security_asset_name` is a compile-time parameter of four validators, and each copy is read independently:

Validator	Parameter row	Reads it in
<code>global_state.global_state_spend_validator</code>	7	<code>no_concurrent_security_mint</code> ; the <code>MintSecurity</code> net
<code>minting_authority.minting_authority_validator</code>	10	the <code>asset-name</code> allowlist <code>only_permitted_assets_minted</code> ; <code>minted_amount</code>
<code>transfer_logic_script.transfer_logic_validator</code>	8	the sender and destination party folds
<code>third_party_transfer_logic_script.third_party_transfer_logic_validator</code>	9	the destination party fold

The README “Parameter order” section marks each of the four as “Operator choice” independently. The only stated constraint is that the name must not carry the `(100)` prefix. The “must receive the identical value” warning applies only to the two issuance-policy slots of row 10.

The supply cap lives solely in the GlobalState `MintSecurity` branch:

```

1 let minted_amount = quantity_of(self.mint, issuance_policy_id, security_asset_name)
  -- :306
2 let remaining_amount = input_datum.mintable_amount - minted_amount
  -- :311

```

If the authority (row 10) was compiled with name `A` and the GlobalState validator (row 7) with name `B`:

- The authority’s allowlist admits a mint of `A` as “the supply”.

- `MintSecurity` computes `minted_amount = quantity_of(mint, policy, B) = 0`, so `remaining_amount == mintable_amount`, the rebuilt datum equals the input datum, and the branch passes.
- Every other `GlobalState` branch's `no_concurrent_security_mint` guard also reads `B` and therefore stops forbidding mints of `A`.

`A` can then be minted indefinitely with `mintable_amount` never decremented. Because the CIP-113 base layer conserves the policy, `A` is a fully programmable token that lives in the base and dispatches to the transfer validators. If the ordinary transfer validator was compiled with `B`, an `A`-only transfer produces no parties, so no denylist or KYC checks run; pause and deactivation checks still apply. If compiled with `A`, everyday compliance works but the cap does not. The seizure validator also requires `at_least_one_seized_input` carrying its configured name, so an `A`-only seizure with that validator compiled for `B` fails closed rather than bypassing all gates.

Nothing on chain can detect the mismatch. The `(100)`-prefix check at registration and the two-slot identity warning do not cover it. The condition requires the operator to misconfigure their own deployment, the same class as the “genesis placement” and “permissive-script credential” deployment risks already listed under *What is still open* in `documents/security/security-fixes.md`.

Impact

- **Supply cap silently disabled.** A mismatch between rows 7 and 10 lets the security asset be minted without bound, with the `GlobalState` datum never recording it.
- **Ordinary transfer gates may see no parties.** A mismatch between rows 8 and 10 lets an `A`-only ordinary transfer omit denylist and KYC checks. A mismatch between rows 9 and 10 instead blocks an `A`-only seizure at `at_least_one_seized_input` (`validators/third_party_transfer_logic_script.ak:168-185`).
- **Not detectable on chain.** No validator compares the four copies, and the deployment checklist has no post-genesis check for it.

Recommendation

1. **On chain, in `MintSecurity`:** require that every asset name minted under `issuance_policy_id` is either `security_asset_name` or carries the CIP-68 protected prefix. A mismatched authority minting `A ≠ B` is then rejected by the `GlobalState` validator instead of slipping past the cap. The authority already enforces the same shape from its side; mirroring it here makes the two compile-time copies of the name check each other on the first mint. Cost: one fold over the one- or two-entry mint dict per mint.
2. **Off chain:** the deploy script should assert that the `security_asset_name` argument applied to rows 7, 8, 9 and 10 is byte-identical, exactly as the README already prescribes for the two issuance-policy slots. Add a post-genesis smoke test: a first mint whose `GlobalState` output datum does not decrement `mintable_amount` must be rejected.

Measure 1 does not help the transfer-side mismatch (rows 8 and 9 against row 10); only measure 2 does.

Add a negative test for an over-cap mint (`minted_amount > mintable_amount`) in `validators/global_state.ak`, the single most important assertion of the branch; all current mint fixtures use 100 against 1,000,000.

Related Findings

- Finding **F204**: registry fields 3 and 4 are not pinned to the compiled transfer-logic hashes at registration. The same fail-open deployment class, at registration.

[L 204] Registry fields 3 and 4 are not pinned to the compiled transfer-logic hashes at registration

Level	Severity	Status
2	Low	Acknowledged

Category: Deployment / configuration

Affected File: `validators/minting_authority.ak`

Description

Every registered programmable token has exactly one node in the CIP-113 registry, whose datum is a `RegistryNode` with seven fields (`../cip113-programmable-tokens/lib/registry_node.ak:51-80` at `9db7e06`). The base layer reads the logic-script credential for each path straight out of that node:

Index	Field	Read by the base layer to dispatch
0	<code>key</code>	the token's policy id
1	<code>next</code>	linked-list link
2	<code>minting_logic_script</code>	mint and burn
3	<code>transfer_logic_script</code>	ordinary transfers
4	<code>third_party_transfer_logic_script</code>	seizure / forced transfer
5	<code>unfracking_logic_script</code>	unfracking; this deployment pins it to <code>VerificationKey("#")</code> , meaning forbidden
6	<code>global_state_cs</code>	this token's GlobalState NFT policy

Fields 3 and 4 are where this module's compliance logic is attached to the token. Whatever hash sits in field 3 is the script the base layer requires a withdraw-0 from on every ordinary transfer (`../cip113-programmable-tokens/validators/programmable_logic/transfer.ak:249-257`), and field 4 plays the same role for seizure (`../cip113-programmable-tokens/validators/programmable_logic/third_party.ak:24-29`). CIP-113 binds `key` cryptographically to field 2, but leaves fields 3 and 4 free.

`verify_registration_structure` in `validators/minting_authority.ak` checks those two fields only for the `Script` constructor, not for the value:

```
1 expect transfer_logic: Credential = transfer_logic_data aiken
2 expect Script(_) = transfer_logic      -- :660, constructor only
3
4 expect third_party_logic: Credential = third_party_logic_data
5 expect Script(_) = third_party_logic    -- :667, constructor only
```

The contrast with the two fields immediately after is the sharpest evidence: fields 5 and 6 are pinned to exact values in the

same function, with `expect node_unfracking == VerificationKey("#")` at :677 and `expect node_global_state_cs == global_state_policy_id` at :685. Of the registry fields this deployment cares about, only 3 and 4 are checked loosely.

A mis-registered hash, from a typo or a wrong build, therefore directs the affected transfer or seizure path to a script other than this module's intended KYC, denylist and pause logic. Registration does not reject the hash mismatch.

Unlike the upgrade path, which legitimately needs the freedom to point at a new validator, registration has no reason to leave the value open: the authority is compiled last, after both transfer hashes are known, and already takes `denylist_script_hash` and `power_user_list_script_hash` as parameters.

This is the same fail-open deployment class as Finding **F203**, at registration rather than at compile time.

Impact

- **Compliance logic can be absent from genesis.** A wrong field 3 directs ordinary transfers to the wrong script; a wrong field 4 does the same for seizure. A permissive replacement can accept the affected operation without the intended denylist, KYC or pause checks.
- **No on-chain equality guard at registration.** The deployment checklist currently has no post-registration verification of the registry node's fields.
- **Recoverable before the upgrade lock.** `UpgradeRegistryNode` permits the admin to replace fields 3 and 4 without invoking the incorrectly registered transfer scripts, provided `deactivated == False` and `upgrades_locked == False` (`validators/minting_authority.ak:300-418`). Recovery corrects future dispatch; it does not undo transfers already accepted by an incorrect script.
- **Stuck after the upgrade lock.** `LockUpgrades` sets the one-way `upgrades_locked` flag (`validators/global_state.ak:562-578`). Subsequent `UpgradeRegistryNode` calls reject, so an incorrect registration can no longer be corrected through this path.

Low severity reflects the operator-misconfiguration prerequisite and the available recovery before locking. A permissive wrong script can omit compliance checks; an unavailable or rejecting script can instead block the affected path. Neither outcome should be described as necessarily permanent at registration.

Recommendation

Add `expected_transfer_logic_script_hash` and `expected_third_party_logic_script_hash` parameters to `minting_authority_validator` and compare them in `verify_registration_structure` on `RegisterMint` and `RegisterStructural` only.

At minimum, add "verify registry-node fields 3 and 4 against the compiled hashes on chain" to the README's initialisation step. Repeat that verification and smoke-test transfer and

seizure behavior before `LockUpgrades` ; correct any mismatch with `UpgradeRegistryNode` while the protocol is active and upgrades remain unlocked.

Related Findings

- Finding **F203**: `security_asset_name` is compiled into four validators with no on-chain coherence check.

[L 205] GlobalState datum caps are per-field, not aggregate

	Level	Severity	Status
	2	Low	Acknowledged

Category: Resource bounds / Availability

Affected Files:

- `lib/constants.ak`
- `validators/global_state.ak`
- `documents/security/security-fixes.md`

Description

Three constants bound the growth of the `GlobalState` datum:

Constant	Value
<code>max_trusted_entities</code>	64
<code>max_trusted_entity_metadata_bytes</code>	512
<code>max_security_info_bytes</code>	4096

Each cap is enforced independently at genesis and on the branches that can grow its field. No branch checks the serialized size of the complete continuing `GlobalState` datum.

At all three caps, the datum reaches **39,186 bytes at the validator level** against a 16 KiB maximum transaction size. The comment in `lib/constants.ak` says the bounds sit far below the transaction-size limit, which is true per field but false in combination.

The existing tests exercise the caps in isolation. For example, the many-entities fixture uses tiny metadata values, while the metadata-size tests use only one entry.

Impact

Every `GlobalState` spend must re-emit the full datum inline in the continuing output. As it grows, the datum crowds out the rest of the transaction and squeezes the most expensive branches first, especially `MintSecurity`, which also carries destination actions and KYC data.

This remains Low because growth is admin-controlled, self-limiting, and recoverable. Importantly, the 39,186-byte figure is a validator-level bound only and **does not prove a ledger-reachable lockup**: every growing spend must carry the full continuing datum in its own output, so the datum can never exceed what a single transaction can write. Near-limit operation degrades headroom for the most expensive branches first, but `RemoveTrustedEntity` (whose own transaction carries a *smaller* continuing datum) shrinks it back.

Recommendation

- Add an aggregate serialized-size check on every branch that can grow `GlobalState`, using `bytearray.length(builtin.serialise_data(expected_output_datum))` against a transaction-budgeted cap.
- Keep the per-field caps, but correct the `lib/constants.ak` comment so it no longer implies their combination fits under the transaction-size limit.
- Add a regression test combining `max_trusted_entities` entries with `max_trusted_entity_metadata_bytes` metadata and max-size `security_info`.

Related Findings

- Finding **F202** covers a separate inline datum size-bound gap on mint-created security-token UTxOs.

[L 206] PauseTransfers can churn the GlobalState UTxO and race admin recovery

	Level	Severity	Status
	2	Low	Acknowledged

Category: Availability / Role management / Design limitation

Affected Files:

- `validators/global_state.ak`
- `validators/power_users.ak`
- `validators/denylist.ak`

Description

`PauseTransfers` lets any power user with `can_pause` spend the single GlobalState UTxO and set `transfers_paused` to the boolean carried in the redeemer. The branch checks the power-user role and signature, but it does not require the requested value to differ from the input datum.

As a result, a `can_pause` operator can repeatedly spend and recreate GlobalState, either as byte-identical no-ops or by alternating pause/unpause. The transaction changes the state UTxO reference each time.

This matters because emergency recovery also depends on the same GlobalState UTxO. `DeactivateContract` requires `input_datum.transfers_paused`, so deactivation must be submitted against a paused pre-state. Power-user and denylist administration paths also read GlobalState as a reference input to resolve the current admin or power-user list policy. A competing `PauseTransfers` spend can make those transactions stale before they land.

Impact

A rogue or compromised `can_pause` key can create a fee-bearing liveness race around the protocol's central control UTxO. The contracts do not give either competing transaction priority and impose no maximum recovery time. Repeated churn therefore may delay recovery for an unbounded duration under adverse transaction ordering; retrying is an operational response, not an on-chain guarantee that recovery will eventually land. No unauthorised party can move holder tokens, mint supply, or bypass KYC/denylist checks.

Note that **rejecting no-op writes does not fix the contention**: an operator can alternate pause/unpause and churn the UTxO just as cheaply. The race is a design limitation of recovery paths that read the contended UTxO as a reference input, inherent to the UTxO model. There is no clean one-line code fix.

Severity stays **Low** rather than Informational because the contended actions are precisely the emergency levers (deactivation and role revocation), and the adversary is exactly the compromised-operator scenario those levers exist to handle. Downgrade to Informational only if the team signs off the residual race as accepted with the resubmit-until-it-lands runbook below.

Recommendation

- Consider direct emergency deactivation without a paused-state precondition. This removes the need to first capture a particular pause state and can shorten recovery, but it still contends for the same GlobalState UTxO and therefore does not guarantee liveness.
- Direct admin pause authority may help the admin restore the required state, but by itself it does not remove `DeactivateContract`'s paused-state prerequisite: the admin must still land a pause transition and then a deactivation transaction, both against the contended GlobalState UTxO.
- If operationally important, allow role-revocation flows to coordinate with a spent GlobalState location instead of only a reference input, so a successful recovery transaction atomically stabilises the control state. It must still win the GlobalState contention race once.
- (Cosmetic only) Reject no-op pause writes by requiring `transfers_paused != input_datum.transfers_paused`. This removes the state-invisible churn variant but does not address the contention: alternating writes churn identically.
- Alternatively, record the residual race as accepted and use monitoring plus resubmission as the operational response, explicitly acknowledging that the contracts provide no maximum recovery time or eventual-success guarantee.

Related Findings

- Finding **F105**: the same `PauseTransfers` actor can also attach a reference script while churning.

[L 207] Authorized seizure applies to script-held tokens without consent, and the resulting accounting divergence is undisclosed

Level	Severity	Status
2	Low	Acknowledged

Category: Undisclosed policy / Integration risk / Third-party and seizure path

Affected File: `validators/third_party_transfer_logic_script.ak`

Description

CIP-113 leaves holder scope to the module. Per `03-CONTROL-SCOPE-AND-ADMIN-AUTHORITY.md` §2.4, the only on-chain signal of who holds a token is the stake credential: a `VerificationKey` is a user wallet, while a `Script` is ambiguous, since a smart wallet and a lending pool are indistinguishable as credentials. The document states plainly that which holders are seizable is a module decision, not a framework rule, and that no single rule is correct for every script. §2.5 then offers two patterns, an issuer allowlist or consent via the script's own `withdraw-0`, **for a module that wants to respect script-owned positions**. Neither is required, and unrestricted authorized seizure is a policy the framework permits.

This module permits it. `third_party_transfer_logic_validator` touches `self.inputs` in exactly two places, neither of which reads an address: `at_least_one_seized_input` at `validators/third_party_transfer_logic_script.ak:174-183` counts `value`, and `no_protected_tokens_spent` at `:195-203` inspects asset names. The only `stake_credential` read is on outputs at `:128`, collecting destinations to vet. A retail wallet and a lending pool are therefore treated identically. Consistent with that, `authorised_stake_cred`, the base layer's single definition of owner consent, is invoked from `programmable_logic/transfer.ak:94` and `programmable_logic/unfracking.ak:169` and is absent from the entire third-party path, which is correct design for a seizure mechanism.

What follows from that policy is a divergence between a contract's stored state and its actual value. On a seizure the base layer enforces per pair, at `programmable_logic/third_party.ak:193-195`, that `output.address`, `output.datum` and `output.reference_script` equal the input's, while the subject policy's tokens on that pair may be decreased or removed entirely. `README.md:288-293` records that pairing rule as a custody guarantee; what it does not draw out is what the rule implies for a holder whose datum encodes a balance. Take a lending protocol holding 1000 units as collateral in a PLB UTxO staked to the pool's script, whose datum records `collateral: 1000`. An operator seizes it:

	Before	After
Pool datum	<code>collateral: 1000</code>	<code>collateral: 1000</code> , preserved by the base layer
Actual value	1000	0, extracted

The stale datum is not speculative; it follows directly from datum equality combined with a permitted token decrease. What the pool then does with that stale figure is implementa-

tion-specific and is not demonstrated here. A contract that re-derives balances from `value` detects the shortfall; one that computes a payout from its datum does not. Detecting the shortfall is also not the same as resolving it, since outstanding debt and loss allocation among the pool's participants remain open questions the contract may have no mechanism to answer.

The pool cannot intervene in either case. Spending is authorized by `programmable_logic_base` rather than by the stake credential, and the third-party path skips `authorised_stake_cred`, so the pool's validator does not run.

Impact

The risk is asymmetric: it falls on integrators rather than the issuer. Every invariant this protocol owns still holds after such a seizure, since supply is capped, value stays in the PLB, conservation reconciles and destinations are still vetted. That asymmetry is most likely why prior passes did not surface it, since each asked whether this protocol could be broken rather than whether it could complicate something built on it.

Severity is Low, as a conditional integration risk. Two components are unverified and should not be presented otherwise. Whether stale data becomes financial loss depends on a specific integrator's implementation, and no affected integration has been identified. Likelihood is unknown, since no deployment evidence was available as to whether script-staked holders exist or are planned. This repository's first-class support for `Script` credentials, through the dedicated `credential_type_script` byte in the KYC payload and the constructor byte in `membership_leaf_key`, shows the holder class is anticipated, but it does not establish that any such holder exists, and on its own it does not justify a higher rating.

If unrestricted authorized seizure is confirmed as intended policy and no affected integration is demonstrated, Informational is a reasonable rating.

What holds regardless of rating, and is the substance of the finding: the authority is real and its accounting consequence follows directly from the base layer's rules, but the consequence is not spelled out for the parties it affects. The README documents datum preservation during seizure, but does not explicitly explain that seizure from script-held positions can invalidate balances recorded in their datums. The missing element is the integration warning, not the mechanism.

Recommendation

Document the authority and require integrations to handle it. Suggested wording for the client-facing documentation:

Authorized seizure also applies to tokens held under smart-contract stake credentials, without that contract's consent. The token balance can decrease while its datum remains unchanged, potentially invalidating an integrator's accounting. We recommend

explicitly documenting this authority and requiring integrations to handle seizure-induced balance shortfalls.

In the repository, record the same point where a maintainer will meet it: a comment beside `at_least_one_seized_input` noting that holder scope is a §2.4 decision this deployment has made deliberately, a line in the README's seizure section, and an `invariants.md` entry making INV-16's holder-side scope explicit. Pin the behaviour with a test, so that a future change to it is deliberate rather than accidental.

A consent gate under §2.5 remains available but needs deeper design work and is not proposed here, for two reasons beyond the policy question. It would let the owner script approve or reject extraction, but would not relax the base layer's requirement to preserve the paired datum. An approving contract therefore accepts the resulting shortfall rather than reconciling it, and the divergence is prevented only where the owner rejects. Rejection is also the likely default for any contract not written with CIP-113 seizure in mind, since a generic withdraw-0 approves only when its internal invariant holds, typically value in equals value out, which a seizure deliberately violates. For a token that must satisfy court-ordered transfers, blocking seizure by default is likely unacceptable. Note that the gate would impose no new registration burden, since a script owning a CIP-113 token already needs a registered stake credential and a working withdraw-0 to spend its own tokens at all, per `programmable_logic/owner.ak:37`.

[I 101] KYC attestation format relies on an unstated single-purpose-key invariant

	Level	Severity	Status
	1	Informational	Acknowledged

Category: Cryptographic hardening / Key management

Affected Files:

- `lib/types/kyc_proof.ak`
- `lib/kyc/verify.ak`

Description

KYC attestations are raw Ed25519 signatures over a fixed 67-byte payload:

```
1 user_pkh(28) | user_kyc_tier(1) | valid_until_ms(8) |
2 security_policy_id(28) | network_id(1) | credential_type(1)
```

text

The verifier checks that the issuer key is listed in `trusted_entity_vkeys`, checks the policy id and network byte, and verifies the signature over the payload as-is.

This is source-accurate but not a concrete vulnerability without an overlapping signer and message format. A collision would require a trusted KYC key to also sign another 67-byte format where this policy id appears at bytes 37 to 64 and the network byte appears at byte 65. No such signer or format is identified.

Impact

Under a single-purpose-key deployment model, there is no known bypass. The remaining risk is operational: the security model depends on trusted-entity keys never being reused for arbitrary messages, non-KYC payloads, or another protocol's compatible 67-byte signing format.

Recommendation

- Add explicit domain separation before upgrades are locked, for example by signing `"CPT-RWA-KYC-v1" || payload` and verifying that same byte string on chain.
- If the current payload is intentionally kept, document the invariant that every trusted-entity Ed25519 key is single-purpose and may only sign this KYC attestation format.

Related Findings

- Finding [F102](#) covers another KYC configuration hardening issue at genesis.

[I 102] Genesis can enable KYC without configuring a proof source

Level	Severity	Status
1	Informational	Acknowledged

Category: Deployment / configuration

Affected Files:

- `validators/global_state.ak`
- `lib/kyc/verify.ak`

Description

`sanitise_initial_datum` accepts genesis states where `requires_sender_kyc` or `requires_receiver_kyc` is enabled while both proof sources are empty:

```
1 trusted_entity_vkeys = []
2 member_root_hash    = #""
```

This is not a KYC bypass. Attestation proofs fail because the issuer key cannot be found in `trusted_entity_vkeys`. Membership proofs fail closed as well: an empty `member_root_hash` traps in `mpf.from_root` before a membership proof can succeed.

Impact

Only the deployer can create this state, and it grants no attacker privilege. The impact is deployment liveness: KYC-gated paths are unavailable until an admin adds a trusted attester key or updates the membership root.

With receiver KYC enabled, ordinary registration destinations fail closed when no proof source is configured. Initial supply can still be minted to the authorizing power user because that destination is exempt from receiver KYC.

Recommendation

- Add a deployment check that refuses genesis with any KYC flag enabled unless at least one trusted attester key or a valid membership-root channel is configured.
- Do not treat a 32-byte `member_root_hash` as proof that holders can verify. It only proves that a channel was configured; deployment tooling should also confirm that the corresponding off-chain membership tree and proof distribution exist.

Related Findings

- Finding **F101** covers a separate attestation key-management invariant.

[I 103] KYC attestation field bindings are not pinned by any test

Level	Severity	Status
 1	Informational	Acknowledged

Category: Test coverage / assurance

Affected Files:

- `lib/kyc/verify.ak`
- `scripts/kyc-test-vectors.mjs`
- `documents/security/security-fixes.md`

Description

`documents/security/security-fixes.md` §9 (“Credential type was erased throughout”) states the fix is “Pinned by `attestation_fails_when_credential_type_mismatches` and `attestation_fails_when_script_payload_used_for_a_key`”. Fix #10 makes the analogous claim for the membership variant.

Every `attestation-path` negative test in `lib/kyc/verify.ak` (`attestation_fails_when_user_pkh_mismatch`, `_tier_is_invalid`, `_ttl_expired`, `_upper_bound_is_infinite`, `_security_policy_mismatch`, `_network_id_mismatch`, `_credential_type_mismatches`, `_script_payload_used_for_a_key`) builds its proof with signature: `test_invalid_signature`. The result of `verify_attestation_proof` is

```
1 and { lengths_ok, fields_ok, issuer_ok, sig_ok } -- lib/kyc/verify.ak:119-124
```

so with an invalid signature every one of those tests passes whether or not the field check it is named after exists. The only genuine-signature negative, `attestation_genuine_signature_fails_when_payload_is_altered`, alters the payload, which isolates the signature check, not a binding. Validator-level tests in `validators/transfer_logic_script.ak` and `validators/minting_authority.ak` use either an all-invalid `dummy_proof` or the genuine proof for the correct party; none presents a genuine attestation for the wrong party, policy, network, or an expired TTL.

Evidence

Deleting the three lines

```
1 payload_user_pkh == utils.credential_hash(expected_credential),
2 payload_policy == security_policy_id,
3 payload_network == network_id,
```

from the `fields_ok` block of `verify_attestation_proof` leaves `aiken check` at 357 passed, 0 failed, identical to the pristine tree. Replacing `ttl_ok` with `True`, or deleting `payload_tier != tier_invalid`, gives the same result.

The membership variant is structurally safer: it derives the leaf key from the expected credential (constructor and hash), and the leaf value from the proof’s TTL plus the expected

security policy and network. These values must match the leaf committed to by the trusted Merkle root, so removing a redundant proof-pkh comparison alone does not remove the credential binding. `membership_genuine_root_rejects_other_credential` exercises that leaf-key binding. This structural protection does not establish complete regression coverage: the recorded TTL mutation causes `membership_fails_when_ttl_expired` to trap on `from_root("#")`, which is not an isolated check of expiry against a valid membership root.

Impact

The attestation verifier carries the compliance perimeter’s strongest single claim: an attestation cannot be replayed for another holder, another token, another network, or after expiry. The record says these properties are regression-pinned; they are not. A regression that reintroduces defect #9, or removes the cross-policy or cross-network replay protection, or the TTL check, on the attestation path passes CI today.

The bindings themselves are present and correct at the reviewed revision. The exposure is to future edits, such as a “tidy the `and` block” change, that would pass unnoticed.

Recommendation

Test-only; no validator change.

1. Extend `scripts/kyc-test-vectors.mjs` to emit genuinely signed payloads for: wrong pkh, `credential_type = 0x01` for a key holder, `tier = 0x00`, `valid_until_ms < tx_upper`, wrong `security_policy_id`, wrong `network_id`. Assert each is rejected against the expected credential, policy and network with the genuine issuer key in the TEL. One test per field.
2. Add one validator-level test on the transfer path that supplies the genuine destination attestation in the source action slot (and vice versa) and asserts rejection.
3. Correct the “Pinned by” sentence in §9 once the tests exist.

Additional Observations

- `beBytes()` in `scripts/kyc-test-vectors.mjs` silently truncates values of $256^{\wedge}\text{width}$ or more (for example `tier` set to 256 becomes `0x00`); the on-chain encoder traps instead. Dev tool only, but an attester that copies it would emit unnoticed bad payloads.
- No off-chain MPF vectors exist; every membership positive control uses a single-leaf trie with `mpf_proof: []`, so `Branch` / `Fork` / `Leaf` step decoding from redeemer `Data` is never exercised in this repository.
- Tier is only checked `!= 0x00`; bytes `0x04–0xFF` are accepted and no ordering exists (`lib/types/kyc_proof.ak`). Documented as v1 behaviour. The membership variant carries no tier and no issuer identity, so the two variants stop being interchangeable the day tier is enforced.

[I 104] Denylist metadata is unbounded

	Level	Severity	Status
	1	Informational	Resolved

Category: Resource bounds / Availability

Affected Files:

- `lib/types/denylist.ak`
- `validators/denylist.ak`
- `lib/denylist/absence.ak`

Description

`Denylist` nodes carry a free-form `metadata: Data` field for off-chain indexing and auditability. On-chain validators do not inspect the field semantically, but it remains part of the inline datum carried by each denylist linked-list node.

The `AddToDenylist` and `RemoveFromDenylist` branches only check that the node datum decodes as `Denylist`. They do not bound the serialized size of `metadata`.

Transfers and mints prove denylist absence by referencing a covering denylist node. The transaction builder cannot freely choose an arbitrary small node: the covering node is determined by linked-list ordering, where the cited bounds must cover the checked credential hash.

Impact

An `is_admin` power user can insert a denylist node with metadata that is unbounded by these validators, subject to ledger limits at creation time.

Correction to the previously claimed impact: transactions that merely *reference* the covering node do **not** serialize its datum in the transaction. Under [CIP-31](#) only the outref is serialized. A larger referenced datum therefore causes no additional fee through serialized transaction size. The remaining considerations are:

- **Potential execution-unit cost:** resolving a larger datum may increase evaluation cost if context decoding or validator-side traversal processes those bytes. This is an unverified hypothesis, not an established size-proportional charge: the read path passes `metadata` to the covering-node callback as opaque `Data` and ignores it. Benchmarking is required to determine whether ordinary flows pay any material delta.
- **The node's own min-ADA** (paid by the inserting `is_admin`).
- **Transaction size only on list mutations** that reproduce the node as an output: its own creation, and any insert that uses it as the anchor.

No denylist, KYC, or supply check is bypassed, and the actor is an authorised sanctioning role. Informational until benchmarking shows a material execution-cost impact on ordinary flows; promote to Low only with that evidence.

Recommendation

- Add a serialized-size bound for `Denylist.metadata` in the `AddToDenylist` callback, mirroring the existing `GlobalState` metadata caps, which is cheap and removes the question entirely.
- Benchmark a transfer citing a covering node with near-limit metadata (cpu/mem delta versus a minimal node) so the execution-cost impact is measured rather than assumed; revisit severity on the result.

Related Findings

- Finding **F202** covers unbounded mint-created holder datums.
- Finding **F205** covers aggregate `GlobalState` datum growth (where, unlike here, the datum *is* carried in every continuing output).

Resolution

`AddToDenylist` now limits serialized `Denylist.metadata` to 512 bytes. The boundary tests accepting metadata at the cap and rejecting metadata one byte over the cap passed at verified revision `7dbc40051a7e840f09ece77d9bf187cef768fd14` with Aiken 1.1.23.

Resolved in commit hash `600aa2ba8ca5de34aade675e33de1dc9e877eb56`

[I 105] GlobalState outputs can carry an unnecessary reference script

	Level	Severity	Status
	1	Informational	Resolved

Category: Resource bounds / Hardening

Affected File: `validators/global_state.ak`

Description

Every `GlobalState` spend re-emits a continuing output selected by `global_state_output_index`. The shared preamble preserves the output address, the GlobalState NFT, non-ADA value, and the expected datum, but it does not check whether the new output carries a reference script.

The genesis mint validator likewise locates and validates the initial GlobalState output without checking its `reference_script`. As a result, the deployer can attach one at genesis, and any authorised GlobalState spender can attach or replace one during a later state transition. This includes admin actions and the `PauseTransfers` branch available to a `can_pause` power user.

Impact

This does not bypass KYC, denylist, supply, pause, or deactivation checks. After genesis, the actor must already be authorised to spend GlobalState. An extra reference script can be removed by a later authorised GlobalState spend only while GlobalState remains active; one present on the terminal deactivation output cannot subsequently be removed.

The practical concern is avoidable resource overhead. GlobalState is consumed by mint and burn transactions and is commonly referenced by transfer-side flows; under Conway, reference scripts on a transaction's inputs **and reference inputs** contribute to the minimum fee ([ledger implementation](#)), so an attachment raises the fee of every subsequent transaction that touches GlobalState, plus the UTxO's own min-ADA. Removal remains possible by any later authorised GlobalState spend while the protocol is active: the attachment is avoidable overhead, not a lockup. This remains informational unless benchmarking shows that realistic reference-script attachments cause important flows to fail near ledger limits.

Recommendation

- Require `reference_script == None` both on the initial GlobalState output in `global_state_mint_validator` and in the shared continuing-output invariants.
- Add regression tests showing that neither genesis nor a valid GlobalState transition can attach a reference script.

Resolution

The genesis validator and every continuing GlobalState spend branch now require `reference_script == None`. The genesis and continuing-output tests that reject attached reference scripts passed at verified revision `7dbc40051a7e840f09ece77d9bf187cef768fd14` with Aiken 1.1.23.

Resolved in commit hash `617cf3af93f5260f0aa415926e7daf1c5d7b2258`

[I 106] RotateAdmin does not rotate CIP-68 metadata custody, and LockUpgrades can make a missed handover unrecoverable

Level	Severity	Status
1	Informational	Acknowledged

Category: Key management / recovery / Metadata integrity / Lifecycle

Affected Files:

- `validators/minting_authority.ak`
- `validators/global_state.ak`
- `validators/third_party_transfer_logic_script.ak`

Background: who may update CIP-68 metadata here

CIP-68 does not prescribe an update authority: whoever can spend the UTxO holding the (100) reference NFT rewrites the metadata. Under this deployment's CIP-113 address model that UTxO sits, like every token of the issuance policy, at an address whose payment credential is the programmable-logic base (`no_escape`) and whose inline stake credential is the owner. At registration, `cip68_output_is_pinned` pins that owner's **hash** to `admin_credential_hash` (`validators/minting_authority.ak:571`; doc comment: "the admin is the CIP-68 metadata authority **by construction**"). The pin is `utils.credential_hash(owner)`, hash only with the constructor discarded, so the owner may be written as `VerificationKey(K1)` (consent = K1's signature) or `Script(K1)` (consent = that script's withdraw-O). The recovery design must therefore identify the intended full credential rather than infer it from the hash alone. Call the registered owner credential **K1**.

From then on, the only spend path for that UTxO is the ordinary transfer arm: base-layer owner consent for K1, plus this repo's transfer logic, which, for a metadata-only transaction, checks only pause, deactivation and CIP-68 shape. It does not apply KYC or denylist checks because the party scans are keyed on the security asset, as covered by Haden Finding 01 (`haden/01_transfer-path-reference-nft-compliance-bypass.md`).

Description

Metadata custody is bound to the *credential* K1, not to the *role* "admin":

- **RotateAdmin moves the role, not the custody.** The branch rewrites one GS datum field under dual signature (old + new admin, `validators/global_state.ak:519`). Re-homing the NFT to the new admin means spending the metadata UTxO, which requires K1's consent, in an additional owner-consented transfer. A *separate* transaction is not mandatory: nothing in the current source blocks composing that transfer atomically into the rotation transaction itself while transfers are unpaused (see Recommendation B). The README **does** document this: the outgoing admin must hand the reference NFT over with an ordinary owner transfer (`README.md:385`). So this is a *documented manual dependency*; the finding is that no coercive fallback exists when the manual step fails.

- **Seizure cannot serve as that fallback, twice over.** `no_protected_tokens_spent` (`validators/third_party_transfer_logic_script.ak:195`) hard-rejects any forced transfer spending a protected token (the fix for old finding **F301**), and independently `at_least_one_seized_input` (`validators/third_party_transfer_logic_script.ak:174`) requires at least one spent input carrying `security_asset_name`, which a metadata-only recovery transaction has none of. Verified against the base layer (9db7e06): the base's seizure mechanics would permit moving the NFT; both barriers are this repository's.
- (Unfracking is irrelevant either way: the base unfracking engine is same-owner restructuring by construction (acted tokens may only land back at the pinned owner address), so even an enabled hook could never move custody from K1 to K2.)
- **LockUpgrades closes the last issuer-controlled lever.** Pre-lock, the issuer can deploy a replacement third-party logic and register it via `UpgradeRegistryNode`. Post-lock (`validators/global_state.ak:565`) **no issuer-controlled recovery path exists in this deployment.** (Base-layer governance replacing the protocol-wide third-party delegate remains a theoretical out-of-deployment path: it is the already-documented CIP-113 coordination trust boundary, not an issuer capability.)

While K1 remains usable and the NFT unmoved, the ordinary documented handover works fine. The emergency-upgrade path is only needed *after* K1 is lost or the NFT has been re-homed by a hostile holder.

Concrete failure scenarios

Compromise. K1 leaks. Note the bug grants the attacker no *new* power: holding K1, they already control metadata custody. The issue is that `RotateAdmin` **fails to revoke** it: the issuer rotates to K2 (dual-sig succeeds, since the issuer still holds K1), protecting `GlobalState`, but the metadata UTxO still answers to K1. The handover becomes a race the attacker can win by front-running the issuer's transfer, re-homing the NFT to a credential only they control. Thereafter they can rewrite the token's investor-facing metadata within `Cip68Datum` shape whenever transfers are not paused and the contract is not deactivated. Coercive recovery: emergency registry upgrade pre-lock; nothing issuer-controlled post-lock.

Loss after rotation. Rotation to K2 completes; K1 is then lost or deliberately destroyed (e.g. decommissioning the outgoing key per key-hygiene policy) *before* the handover transaction. Metadata custody is now unreachable: recoverable pre-lock only via an emergency upgrade, permanently frozen (within issuer control) post-lock.

(Loss of K1 *before* any rotation is out of scope: dual-sig `RotateAdmin` then bricks the whole admin role, metadata included. That is a separate, known property of the dual-signature design.)

Impact

No token value, supply, KYC or denylist property is bypassed, and the handover duty is documented. What remains is: (a) the rotation→handover window is racy under exactly

the key-compromise scenario rotation exists for; (b) a missed or failed handover has no coercive fallback; and (c) `LockUpgrades` converts (b) from an incident into permanence within issuer control: the metadata of a live regulated security becomes unrecoverable by the issuer while the token keeps trading. Severity Informational / design sign-off: the custody separation is explicitly documented behavior and no stated guarantee is violated: the useful content is the lock/recovery warning. Raise to Low if the deployment asserts issuer-controlled recoverability of metadata custody as a mandatory guarantee (e.g. an operational register requirement).

Recommendation

Confirm the intended authority invariant first. Choose one of two mutually exclusive models:

1. **Role-bound custody:** the CIP-68 reference NFT must be owned by the active admin. Define the approved owner as a full `Credential`, not only a 28-byte hash: `VerificationKey(K)` and `Script(K)` are different owners under the CIP-113 base. The current `admin_credential_hash` field cannot express that distinction, so either store the full admin/metadata-owner credential in authenticated state or explicitly require and document one constructor, such as `VerificationKey(admin_credential_hash)`. Then compare credentials directly in the programmable transfer logic and make `RotateAdmin` atomically move custody or provide the recovery branch below. During rotation, the spent GlobalState contains the old admin, so the implementation must authenticate the continuing GlobalState output or handle rotation explicitly. It must also define recovery while ordinary transfers are paused.
2. **Custody-separated authority:** a separately approved full credential, such as a metadata multisig script, owns the NFT independently of the operational admin. In that model, `RotateAdmin` neither requires nor performs a metadata handover; custody rotation and recovery belong to the separately defined metadata-authority lifecycle. Do not add an issuer-admin recovery override unless that override is an explicit business requirement. This remains an Informational design sign-off if the separate custody model is accepted.

(A) Role-bound recovery only: a dedicated `RecoverMetadata` branch. If model 1 is selected and issuer-admin recovery is required, simply relaxing `no_protected_tokens_spent` is NOT sufficient: `at_least_one_seized_input` still requires a security-asset input, so a metadata-only recovery would have to piggyback on an unrelated seizure. Add a separate redeemer branch on the third-party validator with a fully pinned transaction shape:

1. authorization by the **current** `GlobalStateDatum.admin_credential_hash` (not merely `can_force_transfer`, since a generic seizure operator should not gain metadata-moving power, even admin-directed);
2. **exactly one** protected-token input and **exactly one destination output** carrying the same policy, asset name and quantity (1); the destination must be well formed

- (`cip68.output_is_well_formed`) and its full owner `Credential` must equal the approved metadata-owner credential selected above;
3. account for the [base-required paired continuing output](#) for the spent metadata UTxO: it remains at the old-owner address, preserves the input datum and reference script, carries at least the input lovelace and preserves every non-acted policy, but no longer carries the protected NFT. No input carries `security_asset_name`, and the transaction changes no other token quantities;
 4. `GlobalState` is read as a **reference input only** (reject the `GlobalStateSpent` form), so a recovery bundled with `RotateAdmin` cannot read the pre-state admin;
 5. the new-owner destination datum is **datum-identical** to the metadata input's (`Data equality`): recovery moves custody, never content (preserving old finding **F301**'s property; the approved owner rewrites afterwards).

Deployment note: this branch only exists via a registry upgrade, and `LockUpgrades` closes that path, so the fix protects a locked deployment **only if deployed before the lock**. Ship it before locking, or accept that locked deployments keep the gap.

(B) Role-bound handover only: conditionally available today; worthwhile defense-in-depth, not a substitute for (A). If model 1 is selected, the current source permits composing the handover into the `RotateAdmin` transaction while transfers are unpaused: the metadata NFT is not `security_asset_name`, so the rotation branch's `no_security_token_spent` does not bite, and the transfer arm may read `GlobalState` as the spent input via `GlobalStateLocation`. However, rotation does **not** itself supply the metadata owner's authorization. The transaction must separately satisfy the CIP-113 base's owner-consent rule for the full credential currently owning the metadata UTxO: a `VerificationKey` owner must sign, while a `Script` owner must execute its `withdraw-0`. An old-admin signature doubles as owner consent only when the current metadata owner is exactly `VerificationKey(old_admin_hash)`; equal hashes alone are insufficient.

Making this fully authorised atomic bundle the runbook default removes the *missed-handover* gap in routine rotations. Its limits remain: it cannot be built after the current metadata owner is lost or refuses consent; a compromised owner can race the bundle by moving the NFT first; and ordinary transfer is unavailable while `transfers_paused`, which is exactly when emergency rotation may be needed. Pair it with (A). If enforcement rather than runbook convention is wanted, requiring the handover inside `RotateAdmin` must account for the full owner credential and the pause coupling above.

(C) Operational measures, to apply immediately, with or without (A)/(B):

1. Under the role-bound model, make the README handover (`README.md:385`) a fully owner-authorized runbook step of every admin rotation (same maintenance window, verified on-chain). Under the custody-separated model, do not couple metadata handover to `RotateAdmin`; maintain a separate custodian-rotation runbook.

2. Pre- `LockUpgrades` checklist (the critical item): assert on-chain that the metadata UTxO's full owner `Credential` equals the approved credential for the selected authority model before signing the lock: the lock is what makes a failed handover permanent.
3. Under the custody-separated model only, consider a dedicated cold multisig **script** stake credential (updates are rare, so this decouples metadata custody from the operational admin key). This is an alternative to role-bound custody, not an additional requirement. Its withdraw-0 must genuinely enforce the intended multisig authorization because the CIP-113 base treats execution of that script as owner consent.
4. Under the role-bound model, extend the README handover note (`README.md:385`) with the lock interaction: a handover missed before `LockUpgrades` , or an outgoing owner credential lost before handover, cannot be remedied by the issuer on chain afterwards. Under the custody-separated model, document its independent recovery procedure and whether locking intentionally removes any issuer override.

Related Findings

- Haden Finding 01 (`haden/01_transfer-path-reference-nft-compliance-bypass.md`): metadata recipients are unvetted (the vetting half of the same design decision; also what makes hostile re-homing unblockable).
- Finding **F301** (fixed): the original seizure-path metadata rewrite; its fix is one of the two barriers that also block coercive recovery. Recommendation (A) preserves its content-integrity property.

End of Report



No Witness Labs | Confidential

Copyright © 2026